PETase AI Bio-Hub

Phone prototype for an AI-engineered PET-degrading enzyme — explanation and results

1. What it is

PETase AI Bio-Hub is an offline Android / browser app that turns the Step 9 notebook of your PETase project into something you can hold and test. It takes a sequence and the model outputs you computed (Tm, binding ΔG, relative activity), then simulates a 24-hour stir-reactor run that depolymerises PET into terephthalic acid (TPA) and ethylene glycol. It also analyses sequences, checks whether a scanned item is PET, exports gene-order and log files, and can push a configuration to a Wi-Fi reactor controller.

Scope. All conversion and yield figures are **simulations**. The phone does not run ESM-2; Tm, ΔG and RMSF are values you type in. Nothing has been validated in a wet lab.

2. The four screens

Screen	What works	Notes
Reactor	Start/pause/reset a live run; heater, set-point 30–75 °C, stir 50–500 RPM, PET load, volume; live chart; Tm-margin indicator; methanol-feed log	Temperature relaxes to the set-point with τ ≈ 1.3 h; sim speed 1/4/12 h per second
Scanner	Camera preview and frame capture; recycling-code (1–7) selection; PET-only verdict; half-conversion time, enzyme mass, 24 h TPA/EG yield	Automatic resin recognition is not implemented — the code is chosen by the user
Mutator	Length, molecular weight, GRAVY, net charge, aliphatic index; substitutions vs wild-type fragment; FASTA import; model-output fields	Tm/ΔG/activity come from your notebook
Report	Live report, export FASTA / CSV / JSON, share, save runs, import samples, HTTP POST to an IoT device	Android app posts natively (no CORS/mixed-content issues)

3. Workflow

- **Design** — run the ESM-2 notebook; note Tm, ΔG, relative activity for the chosen variant.
- **Import** — paste the sequence or load a FASTA/JSON file in the Mutator tab; enter the model outputs; press Use in reactor.
- **Scan** — photograph the item, confirm recycling code 1 (PET) and enter its weight.
- **Simulate** — choose set-point, stirring and load; start the 24 h run and watch conversion and the Tm margin.
- **Export** — FASTA for gene ordering, CSV run log, JSON config; optionally push the config to the reactor controller.

4. The kinetics model

The same equations run in `index.html` and `pipeline/petase_step9.py`:

```
k = 0.045 · (T / 55) · activity · stir(rpm) · stability(T, Tm)
X(t) = min( 100 · (1 − e^(−k·t)), 95.2 ) [% conversion]
stir = min(1, √(rpm / 200)) [assumption]
stability = 1 if T ≤ Tm else e^(−(T − Tm)/3) [assumption]
TPA (g) = load · volume · X/100 · 166.13 / 192.17
```

The first line is the notebook's own rate law. The stir and stability terms are explicit modelling assumptions added so that changing the controls has a physical effect: a wild-type enzyme (Tm ≈ 47 °C) collapses at 60 °C, while the mutant keeps full activity as long as the set-point is below its Tm.

5. Results

With the notebook's values (60 °C, Tm 61.2 °C, 250 RPM, 50 g/L PET, 1 L):

Metric	Value
Conversion at 4 / 12 / 24 h	17.83 % / 44.52 % / 69.22 %
Half-conversion time	14.1 h
TPA released in 24 h (from 50 g PET)	29.9 g
Tm margin over set-point	+1.2 °C (thin: any overshoot above 61.2 °C starts to cost activity)
Wild-type @ 60 °C (Tm 47)	0.5 % — denatured

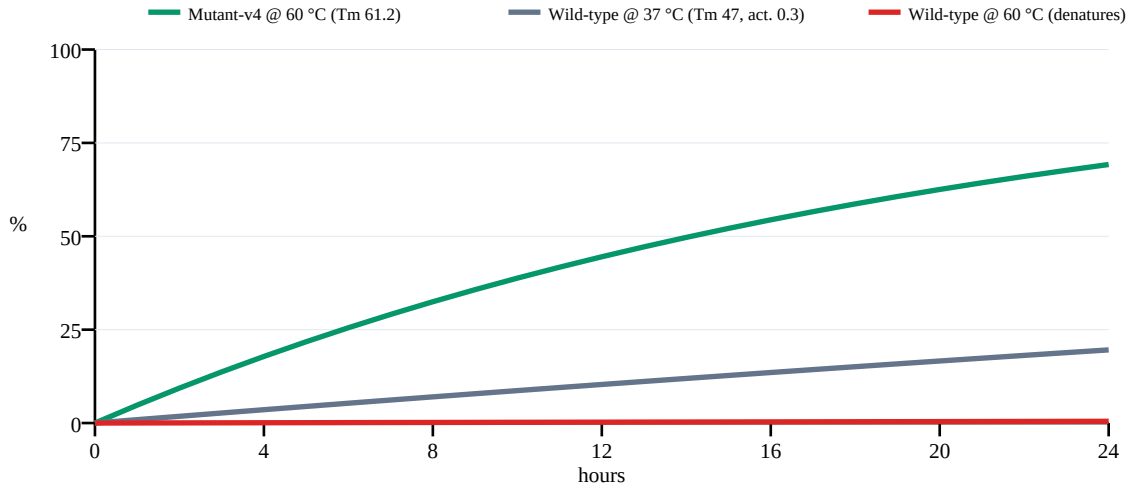


Figure: simulated conversion over 24 h.

6. How the package was checked

- The Python pipeline and the app's JavaScript give the same numbers (69.22 % at 24 h, 17.83 % at 4 h); the app script was executed headlessly with a DOM stub and passed 8 automated checks (kinetics, cap, wild-type collapse, live-run integration, sequence statistics, mutation list).
- The 3-slide deck passed the PPTX validator and was rendered to images for a visual check.
- **Not done here:** the Android project and APK were **not compiled** (the Android SDK could not be downloaded in the generation environment), and the app was not run on a physical phone. Build it with the included GitHub Actions workflow or `build_apk.sh`; any Gradle error will name the exact line.

7. Changes made to your original code

Original	Problem	Now
`conversion = 100.0 * (1.0np.exp(…))`	Syntax error (missing operator)	`100*(1-exp(-k·t·T/55))`
`max(61.2, simulated_triad_distance*17.0)`	Forces “Tm > 60 °C achieved” regardless of the model	Tm is an explicit input; report shows the real margin
Chart showed 78.5 % at 24 h; card showed 69.2 %	Two different numbers for one quantity	Both come from one function
Wild-type Tm 37 °C	Below reported IsPETase values (roughly mid-to-high 40s °C)	47 °C, editable; wild-type activity 0.3 is an assumption
“Catalytic triad locked”, “Cleared for synthesis”, “Low-energy certified”	Not supported by anything the app computes	Removed; status reflects Tm margin only
Scanner showed a fixed “PET #1 Detected (98.4 %)”	Mock result regardless of image	Real camera + user-confirmed recycling code; PET-only logic
Variables from earlier cells (target_sequence …)	Cell not runnable on its own	Standalone script with CLI arguments

8. Limitations to state honestly at the hackathon

- The 20-residue sequences are fragments; real IsPETase is about 290 residues and the catalytic triad (Ser160, Asp206, His237) lies outside them. Order genes only from a full-length design.
- Tm, ΔG and RMSF are model predictions; predicted Tm shifts of this size need experimental confirmation (e.g. differential scanning fluorimetry).
- Real PET flakes are partly crystalline; conversions near 70 % in 24 h at 50 g/L typically need pre-treatment, and enzyme loading is a key unknown (the 0.034 mg/g dose is inferred from your original card).
- Recombinant Pichia production with methanol induction is a laboratory activity with its own safety and regulatory requirements — “home-scale” here means a small bench reactor.

9. Suggested next steps

- Express the variant and run a small plate assay on PET film at 50–60 °C; measure TPA/MHET by HPLC and fit k to replace the assumed constant.
- Measure Tm by DSF to replace the predicted value, and re-enter it in the Mutator tab.
- Add a temperature-sensor feed from the reactor controller so the Reactor tab shows real telemetry.
- Package contents and build steps: see README.md in the zip.